Projet : Modélisation d'une épidémie avec le modèle SIR et variantes

Camille Madier, Hugo Loureiro

12 avril 2026

Table des matières

1 Introduction **2**

2 Modèles et résultats **2**
2.1 Le modèle SIR (Susceptibles, Infectés, Remis) 2
2.1.1 Le modèle 2
2.1.2 Les paramètres et la méthode numérique 3
2.1.3 Le R_0 et le R_t 3
2.1.4 Les résultats 4
2.1.5 Le seuil d'immunité 4
2.1.6 Impact du R_0 : Une analyse par épidémie 5
2.2 Ajout de compartiments (Décédés, Vaccinés) : Le Modèle SIRDV 5
2.2.1 Choix des paramètres 5
2.2.2 Equations du modèle 6
2.2.3 Résultats 6
2.3 Etude avec deux populations (Standard et Vulnérables) dans un modèle SIRD . 6
2.3.1 Choix des paramètres 6
2.3.2 Les équations de la dynamique avec le couplage 7
2.3.3 Résultats 8

3 Preview et conclusion **9**
3.1 Modèle Zombie 9
3.1.1 Implémentation 9
3.2 Modèle Spatiale avec diffusion 9
3.2.1 Modèle SIRD avec diffusion 10
3.3 Epidémie COVID 19 10
3.4 Conclusion 11

4 Annexe **11**
4.1 Codes 11

1 Introduction

Nous étudions ici la dynamique des épidémies.

Nous commencerons par analyser le modèle élémentaire SIR, qui décrit l'évolution de trois compartiments au cours du temps : la population saine, les infectés et les guéris (ou remis).

La modélisation de cette dynamique repose sur des équations différentielles reliant le taux d'accroissement temporel (la dérivée) de ces populations à ces mêmes populations, pondérées par certains paramètres, comme la contagiosité journalière β.

Nous étudierons certaines épidémies notamment grâce aux R_0 et R_t, c'est-à-dire le nombre de contaminations causées par une personne infectée, respectivement au début de l'épidémie et à l'instant t. Ces quantités permettent de caractériser l'évolution d'une épidémie et le seuil d'immunité d'une population donnée.

Le modèle SIR pouvant être enrichi de nombreuses façons, nous l'étendrons ensuite à d'autres compartiments, notamment avec le modèle SIRDV, ainsi qu'à plusieurs types de populations, à travers un modèle SIRD à deux populations.

Par la suite - en cours de développement (cf. preview) - nous étudierons un modèle de type « zombie », puis un modèle zombie avec diffusion, fondé sur une loi de Fick. Nous aborderons ensuite un modèle spatial par région (modèle en réseau) et analyserons l'épidémie de Covid à partir de données réelles.

Enfin, en option et si le temps le permet, nous présenterons des modèles probabilistes ainsi qu'une implémentation fondée sur l'algorithme de Gillespie.

2 Modèles et résultats

2.1 Le modèle SIR (Susceptibles, Infectés, Remis)

2.1.1 Le modèle

Nous analysons ici la dynamique d'une épidémie suivant un modèle SIR décrit par ce système d'équation :

$$\begin{cases} \dfrac{\partial S}{\partial t} = -\beta SI \\ \dfrac{\partial I}{\partial t} = \beta SI - \gamma I \\ \dfrac{\partial R}{\partial t} = \gamma I \end{cases}$$

Avant de commenter les résultats, précisons l'interprétation des équations du modèle et des paramètres associés.

— Le modèle permet de décrire 3 "compartiments" :
  - les sains "S"
  - les infectés "I"
  - les guéris "R"

— nous choisissons de simuler ces classes en pourcentage : leur somme vaut 1 à chaque instant, et chaque classe est comprise entre 0 et 1.

— Le taux quotidien d'infectés $\frac{\partial I}{\partial t}$ est la différence de deux termes :
  - βSI sont les nouveaux infectés par unité de temps. Ce terme s'interprète comme une probabilité jointe : le produit du taux moyen de contamination journalier β, par le taux de personnes infectés I(t), par la probabilité de rencontrer une personne saine S(t).
  - γI sont les nouvelles personnes guéries par unité de temps. Les infectés guérissent pendant un temps $1/\gamma$ au cours duquel ils sont contagieux : chaque jour nous avons ainsi γ fois $I(t)\%$ personnes guéries supplémentaires.

— Ce que "gagne" ou "perd" le compartiment d'infectés I est respectivement perdu et récupéré par les compartiments S et R.

2.1.2 Les paramètres et la méthode numérique

Nous avons choisi pour nos simulations, des valeurs correspondant à des ordres de grandeur communiquées pendant l'épidémie de type COVID :

— L'unité de temps vaut 1 jour.
— La population initiale est de $S(0) = 0.999$, $I(0) = 0.001$ les primos infectés.
— $\gamma = 0.2$ de sorte à avoir une personne contagieuse pendant $1/\gamma = 5 jours$
— R_0 =2,8 le nombre moyen initial de personnes infectées par un malade (**cf. sous-section suivante pour l'analyse de ce paramètre**), ainsi que R_t, son évolution au cours du temps

On détermine β, connaissant les deux paramètres précédents : $\beta = R_0 \times \gamma = 0.58$

Résolution numérique :

Nous avons un système d'équation du type $y_i'(t) = f(y_1, ..., y_n)$, pour lequel se prête donc la méthode d'Euler. Nous implémentons ainsi la méthode qui calcule les dérivées (cf. code) de S(t), I(t), R(t) de sorte, à calculer, itérativement, les valeurs S(t+dt), I(t+dt), R(t+dt) à chaque pas de temps dt.

Nous choisissons de suivre l'épidémie sur 100 jours avec une discrétisation de 1000 pas de temps, soit dt=0,1 jour.

2.1.3 Le R_0 et le R_t

$R_t(t) = \frac{\beta}{\gamma} S(t) = R_t(0)S(t)$ **est caractéristique de la dynamique épidémique** et représente le nombre de personnes infectées en moyenne par un malade.

Avec, pour nos données à l'instant initial $S(t) \approx 1$, $R_0 = \frac{\beta}{\gamma} = \frac{0.558}{0.2} = 2.8$ (conformément aux données communiquées durant l'épidémie de COVID initialement).

Ainsi, quand R_t atteint 1 (puis passe en dessous), l'épidemie est en rémission. Nous vérifions que, pour R_t ainsi définie, cela implique bien - de façon équivalente - une dérivée négative de I_t :

$$\frac{dI}{dt} = \beta SI - \gamma I = I(\beta S - \gamma) < 0$$

soit :

$$I(\frac{\beta}{\gamma}S - \frac{\gamma}{\gamma}) = I(R-1) < 0$$

Ainsi, une dérivée négative de I correspond bien à $R_t(t) < 1$

2.1.4 Les résultats

On voit sur la figure 1 que :

— La population d'infectés croit jusqu'à atteindre un pic au point ou $R_t = 1$. Une personne infecte alors moins d'1 personne en moyenne : l'épidémie est en rémission.

— C'est à ce moment là donc que $\frac{\partial I}{\partial t}$ devient négative et I décroit.

— Ce moment constitue un point d'inflexion pour la population S(t) (et R(t)) qui décroit encore, mais de moins en moins, jusqu'à atteindre sa valeur finale quand $R_t(t) = 0$. L'épidémie est terminée.

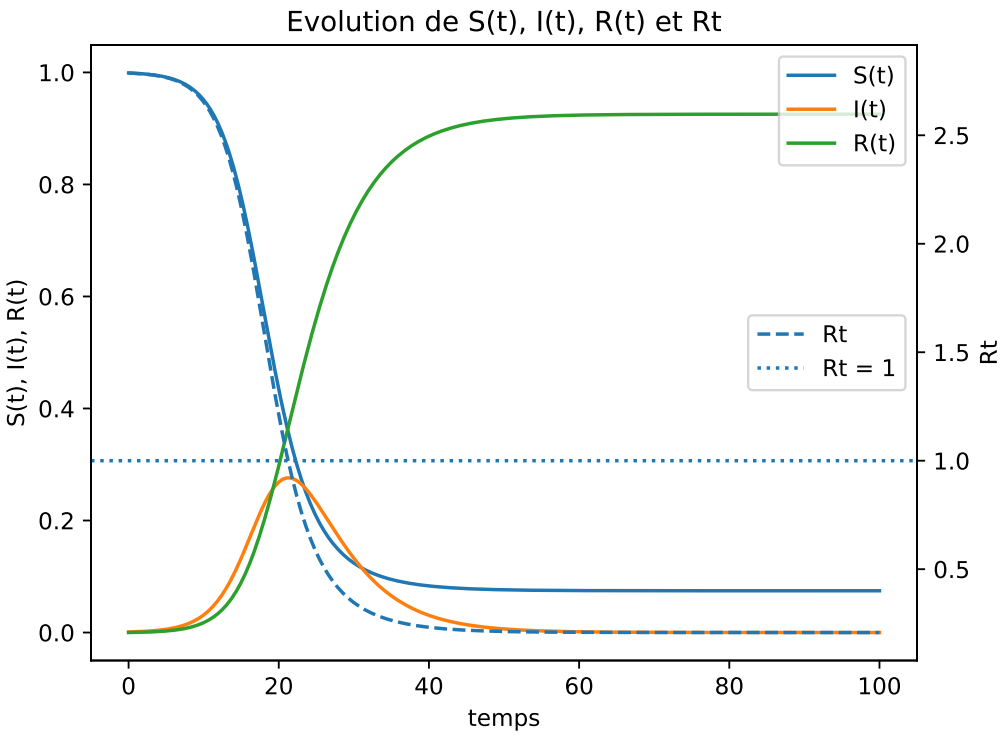


FIGURE 1 – Evolution - en % - de la population saine, infectée, remise, et du R_t, pour une épidémie de Covid

2.1.5 Le seuil d'immunité

On peut calculer le seuil d'immunité théorique d'une maladie donnée, à partir du $R_t(0)$.

Il correspond au pourcentage de la population immunisée (vacciné ou ayant déjà eu la maladie) nécessaire pour que l'épidémie ne puisse pas se propager, partant d'un patient 0.

On l'obtient en considérant l'instant $t = t_0$, auquel $R_t = R_t(0)S(t_0) = 1$ soit lorsque $S(t_0) = \frac{1}{R_t(0)}$. Le seuil recherché correspond alors à la proportion de population immunisée à cet instant :

$$\boxed{\text{Seuil d'immunité} = I(t_0) + R(t_0) = 1 - S(t_0) = 1 - \frac{1}{R_t(0)}}$$

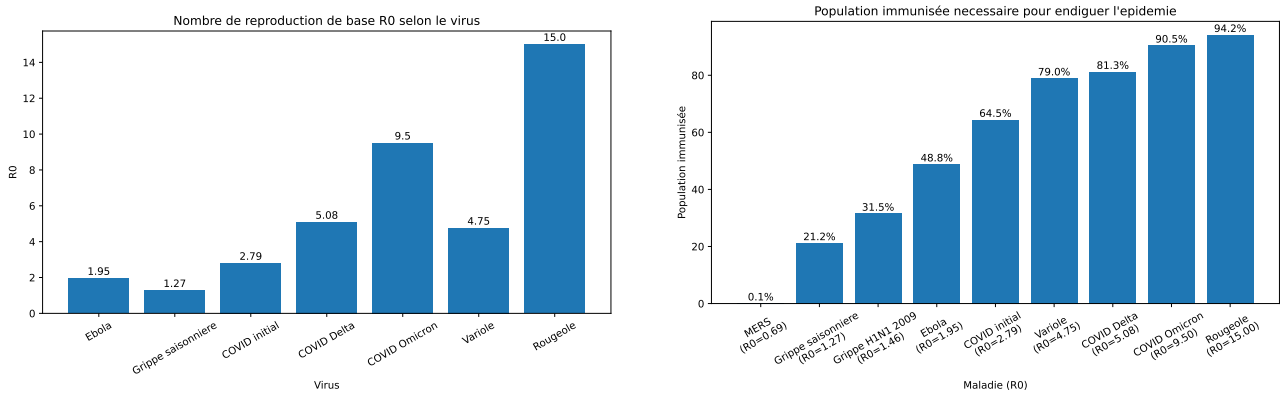
Pour nos simulations numériques de Covid (cf. figure 1), nous trouvons au point $R_t(t_0) = 1, 1 - S(t_0) \approx 1 - 35\% = 65\%$

qui correspond bien à la valeur théorique attendue :

$$\text{Seuil d'immunité} = 1 - \frac{1}{R_t(0)} = 1 - \frac{1}{2.8} \approx 65\%$$

C'est bien le seuil communiqué pendant la première vague d'épidémie par les médias.

2.1.6 Impact du R_0 : Une analyse par épidémie



(a) Donnée de R_0 par maladie

(b) Seuil d'immunité par maladie

FIGURE 2 – Seuil d'immunité en fonction du R_0 (c'est à dire par maladie)

Nous calculons ainsi pour plusieurs épidémie (cf. figure 2) la valeur d'imminuté collective nécessaire pour mettre fin à une épidémie.

Nous vérifions au passage à chaque fois que la valeur de $R_t(0)$ sous jacente correspond bien à la simulation numérique.

Nous choisissons, pour simplifier, de fixer arbitrairement la durée de contagion $1/\beta = 5$ jours comme pour la Covid, et nous implicitons le beta correspondant à partir du $R_t(0)$: $\beta = R_0(\text{épidémie}) \times \gamma$.

Nous effectuons alors la simulation, comme à la section précédente, pour chaque épidémie (cf. code).

2.2 Ajout de compartiments (Décédés, Vaccinés) : Le Modèle SIRDV

2.2.1 Choix des paramètres

— Nous choisissons, pour simplifier, de vacciner uniquement les personnes saines S.

— Nous utilisons les mêmes données que celles de la COVID 19 : une personne est contagieuse pendant 5 jours et peut infecter environ 3 personnes.

— N'ayant pas encore de taux de mortalité μ vérifié, nous choisissons $\mu = 0.002$. De même pour le taux de vaccination, nous choisissons $\omega = 0.01$.

2.2.2 Equations du modèle

$$\frac{dS}{dt} = -\beta IS - \omega S \quad ; \quad \frac{dR}{dt} = \gamma I$$
$$\frac{dI}{dt} = \beta IS - \gamma I - \mu I \quad ; \quad \frac{dD}{dt} = \mu I$$
$$\frac{dV}{dt} = \omega S$$

2.2.3 Résultats

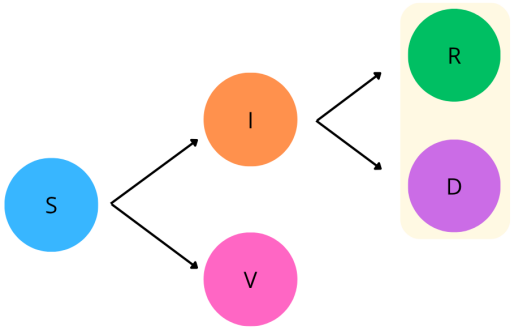


FIGURE 3 – Schéma modèle SIRDV

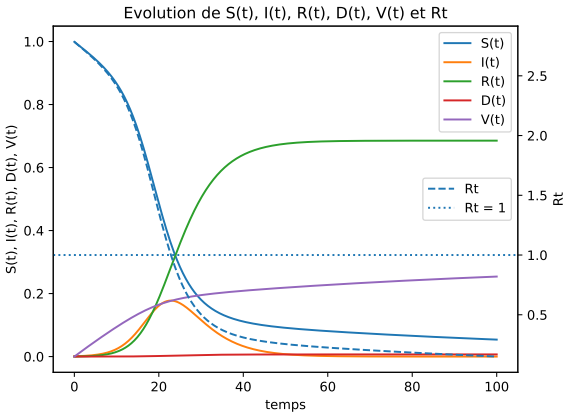


FIGURE 4 – Graphe modèle SIRDV

— β : Coefficient d'infection γ : Coefficient de guérison
— μ : Coefficient de mortalité ω : Coefficient de vaccination

La pente de S : Elle est beaucoup plus raide au début car les gens quittent le compartiment "Susceptibles" pour deux raisons : l'infection ET le vaccin.

Le sommet de I : Le sommet est plus bas : c'est le bénéfice du vaccin.

Taux R_t : La décroissance est plus marquée, permettant d'atteindre rapidement une immunité collective

Conclusion : Ce graphe illustre l'efficacité d'une campagne de vaccination continue pour "aplatir la courbe" et protéger la population sur le long terme.

2.3 Etude avec deux populations (Standard et Vulnérables) dans un modèle SIRD

Nous séparons désormais, dans un modèle SIRD, la population en deux groupes : les jeunes "J" et les vulnérables "V".

2.3.1 Choix des paramètres

— Les $\beta_{J,J}$ et $\beta_{V,V}$ représentent les contaminations intra-groupe, tandis que les $\beta_{V,J}$ et $\beta_{J,V}$ représentent les contaminations inter-groupes :
  - $\beta_{J,J}$: non vulnérable $\rightarrow$ non vulnérable
  - $\beta_{J,V}$: vulnérable $\rightarrow$ non vulnérable

  - $\beta_{V,J}$: non vulnérable $\rightarrow$ vulnérable
  - $\beta_{V,V}$: vulnérable $\rightarrow$ vulnérable
- Les populations initiales J et V sont respectivement de 70 % et 30 %.
- La population J tente de préserver la population V : les coefficients de contamination croisés sont plus faibles que les coefficients diagonaux (cf. code).
- $\beta_{V,V} < \beta_{J,J}$ car les individus vulnérables se protègent davantage.
- $\mu_J < \mu_V$ (taux de décès supérieur pour les "V")
- La dynamique des nouveaux infectés comporte désormais deux termes, correspondant aux contaminations intra-groupe et inter-groupes :
  - Pour J : $S_J\left(\beta_{J,J} I_J + \beta_{J,V} I_V\right)$
  - Pour V : $S_V\left(\beta_{V,J} I_J + \beta_{V,V} I_V\right)$
- Les nombres de reproduction de base $R_{t,J}(0)$ et $R_{t,V}(0)$ deviennent :

$$R_{t,J}(0) = \frac{\beta_{J,J}}{\gamma_J + \mu_J} S_{J,\text{init}} + \frac{\beta_{J,V}}{\gamma_J + \mu_J} S_{V,\text{init}},$$
$$R_{t,V}(0) = \frac{\beta_{V,V}}{\gamma_V + \mu_V} S_{V,\text{init}} + \frac{\beta_{V,J}}{\gamma_V + \mu_V} S_{J,\text{init}}.$$

2.3.2 Les équations de la dynamique avec le couplage

Nous avons alors :

$$\frac{dS_J}{dt} = -S_J\left(\beta_{J,J} I_J + \beta_{J,V} I_V\right)$$

$$\frac{dI_J}{dt} = S_J\left(\beta_{J,J} I_J + \beta_{J,V} I_V\right) - (\gamma_J + \mu_J) I_J$$

$$\frac{dR_J}{dt} = \gamma_J I_J, \qquad \frac{dD_J}{dt} = \mu_J I_J$$

$$\frac{dS_V}{dt} = -S_V\left(\beta_{V,J} I_J + \beta_{V,V} I_V\right)$$

$$\frac{dI_V}{dt} = S_V\left(\beta_{V,J} I_J + \beta_{V,V} I_V\right) - (\gamma_V + \mu_V) I_V$$

$$\frac{dR_V}{dt} = \gamma_V I_V, \qquad \frac{dD_V}{dt} = \mu_V I_V$$

Avec la matrice du couplage :

$$B = \begin{pmatrix} \beta_{J,J} & \beta_{J,V} \\ \beta_{V,J} & \beta_{V,V} \end{pmatrix}$$

2.3.3 Résultats

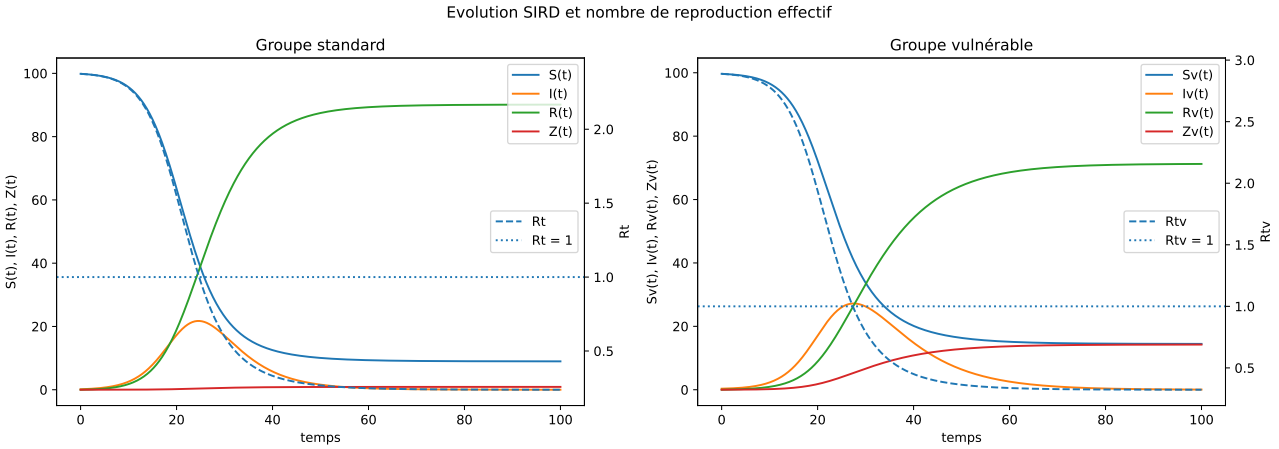


FIGURE 5 – Evolution - en % - de la population saine, infectée, remise, et du R_t, pour une épidémie de Covid

Population	Pop_init (%)	Immunisés	Sains	Décès	tx_pour_immunité	Jour $R_t < 1$
Standard	70.0	90.12	8.97	0.90	60.38	24.8
Vulnérable	30.0	71.23	14.46	14.25	58.47	27.4

TABLE 1 – Résumé des indicateurs par population.

Bilan : comme attendu, les décès sont plus importants chez les V. La population atteint une immunité globale autour du 25ème jour. Quand l'épidémie est terminée, 2/3 des V ont eu la maladie et 90% chez les J.

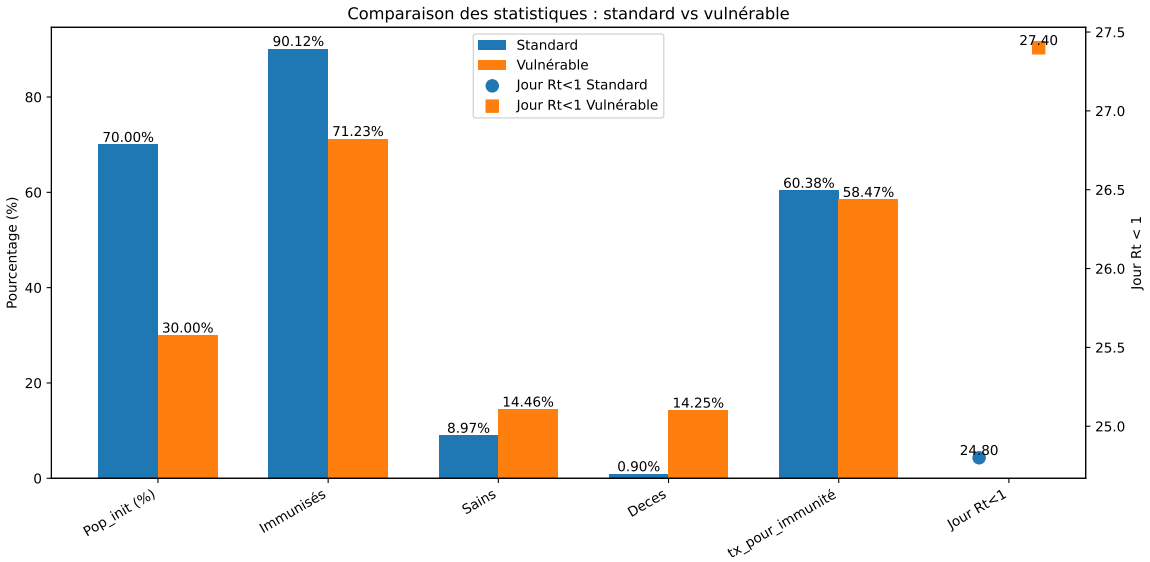


FIGURE 6 – Evolution - en % - de la population saine, infectée, remise, et du R_t, pour une épidémie de Covid.

3 Preview et conclusion

3.1 Modèle Zombie

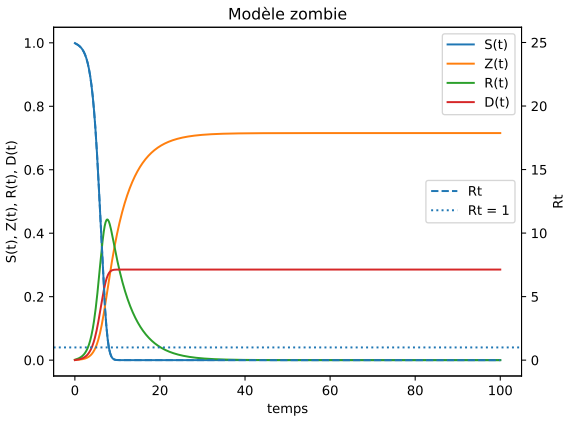


FIGURE 7 – Modèle zombie

Par curiosité, nous avons simulé un scénario de crise extrême basé sur un modèle de type SZR (Susceptibles, Zombies, Removed). La dynamique observée ici est très agressive : la population humaine (S) est annihilée en moins de 10 jours. Nous souhaitons affiner ce premier jet afin de coller aux scénarios rencontrés dans la fiction (The Walking Dead, The Last of Us).

Cela impliquera l'ajustement de paramètres spécifiques comme :

— **Le temps d'incubation ($R \to Z$) :** plus ou moins long selon le mode de transmission (morsure ou infection fongique)

— **Le taux de létalité humaine :** pour simuler une résistance organisée efficace puisqu'une certaine population continue de survivre

— **La réanimation :** l'introduction d'un flux des morts naturels (D) vers les zombies (Z), caractéristique majeure de certaines de ces œuvres.

3.1.1 Implémentation

$$\frac{dS}{dt} = -\mu SZ - \beta SZ$$

$$\frac{dZ}{dt} = \gamma R - \alpha SZ$$

$$\frac{dR}{dt} = -\gamma R + \beta SZ$$

$$\frac{dD}{dt} = \alpha SZ + \mu SZ$$

— α : zombies auxquels on a coupé la tête ;

— γ : délai avant qu'un individu ne devienne zombie ;

— μ : humains entièrement consommés par un zombie ;

— β : humains contaminés par un zombie (morsure).

3.2 Modèle Spatiale avec diffusion

En cours d'aptation pour le modèle Zombie de la section 3.1 (nous appliqerons la diffusion a Z plutot qu'a I) :

3.2.1 Modèle SIRD avec diffusion

(cf. code pour la méthode de dérive et le laplacien.)

Pour prendre en compte la propagation spatiale des infectés, on ajoute un terme de diffusion au modèle SIRD, fondé sur la loi de Fick :

$$\vec{j}_I = -D_I \nabla I,$$

où $I(t, x, y)$ désigne la densité d'infectés.

L'équation de conservation des infectés devient alors

$$\frac{\partial I}{\partial t} = -\operatorname{div}(\vec{j}_I) + \sigma_I = D_I \Delta I + \sigma_I,$$

avec

$$\sigma_I = \beta S I - (\gamma + \mu) I.$$

On obtient ainsi le modèle SIRD avec diffusion :

$$\begin{cases} \dfrac{\partial S}{\partial t} = -\beta S I, \\ \dfrac{\partial I}{\partial t} = \beta S I - (\gamma + \mu) I + D_I \Delta I, \\ \dfrac{\partial R}{\partial t} = \gamma I, \\ \dfrac{\partial D}{\partial t} = \mu I. \end{cases}$$

Le système est étudié sur le carré $[0, \ell] \times [0, \ell]$, avec une condition de flux nul sur le bord :

$$\vec{j}_I \cdot \vec{u} = 0,$$

soit

$$\frac{\partial I}{\partial x}(0, y, t) = 0, \quad \frac{\partial I}{\partial x}(\ell, y, t) = 0, \quad \frac{\partial I}{\partial y}(x, 0, t) = 0, \quad \frac{\partial I}{\partial y}(x, \ell, t) = 0.$$

3.3 Epidémie COVID 19

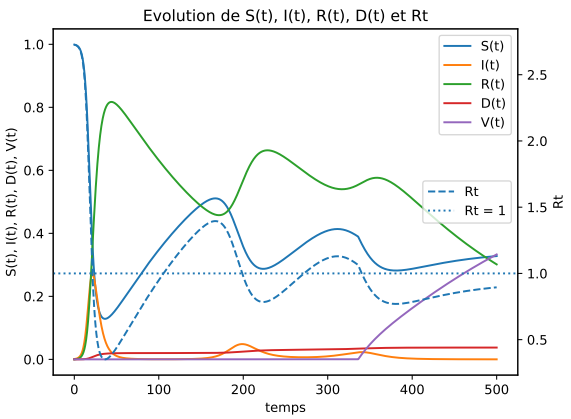


FIGURE 8 – Modèle Covid 19

Ce premier rendu graphique constitue un concept pour la suite de notre étude. Bien que les paramètres actuels soient simplifiés, ce graphe montre notre capacité à simuler des dynamiques avec plusieurs vagues et l'impact d'une campagne de vaccination différée (visible à partir de $t = 340$). À terme, ce modèle sera complexifié pour coller à la réalité de la COVID-19 en intégrant des variables plus fines : prise en compte de la perte d'immunité naturelle (flux $R \to S$) et introduction de nouveaux compartiments pour affiner l'étude. L'objectif final sera de confronter ces simulations aux données réelles de santé publique pour évaluer la pertinence de nos prédictions.

3.4 Conclusion

Le modèle SIR et ses variantes offrent une grande flexibilité (une infinité...) et sont particulierement adaptés à la modélisation d'une épidémie (cf sections précédentes) : "To be continued" avec les preview !

4 Annexe

4.1 Codes

Listing 1 – Code du modèle SIR

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nosPackages.mathUtils as mathUtils

# population initial
S_init=0.999 # personnes suscétibles (départ) en fraction -> 1=100%
I_init=0.001 # personnes infectés
R_init=0 # personnes guérit

# paramètre de contagion
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6
    personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
R0=beta/gamma*S_init # nombre de personnes contaminés par un malade = 3 pour
     le covid

dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

S = np.empty(n)
I = np.empty(n)
R = np.empty(n)
Rt = np.empty(n)

y = np.empty(3)
y[0] = S_init
y[1] = I_init
y[2] = R_init

params={"beta":beta,"gamma":gamma}
model=mathUtils.SIR(params)
deriv=model.deriv_SIR
euler=model.euler

for i in range(n):
    S[i] = y[0]
    I[i] = y[1]
    R[i] = y[2]
    t = temps[i]
    y = euler(t, dt, y, deriv)
Rt = beta/gamma*S # evolution du R0 (contagion pour une personne)


#graphes
```

```python

plt.figure()
plt.title("Evolution de S(t), I(t), R(t) et Rt")

ax1 = plt.gca()
ax1.plot(temps, S, label="S(t)")
ax1.plot(temps, I, label="I(t)")
ax1.plot(temps, R, label="R(t)")
ax1.set_xlabel("temps")
ax1.set_ylabel("S(t), I(t), R(t)")
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(temps, Rt, "--", label="Rt")
ax2.axhline(1, linestyle=":", label="Rt = 1")
ax2.set_ylabel("Rt")
ax2.legend(loc="center right")

plt.savefig("sir_SIR_Rt.pdf", bbox_inches='tight')
plt.show()



gamma = 0.2
dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

# boucle sur beta : de 0.2 à 3.0 , donc de R0=1 à 15
betaTab = np.arange(0.2, 3.0 + 0.2, 0.2)

# vecteurs résultats
R0List = np.empty(betaTab.size)
RtFinal = np.empty(betaTab.size)
RFinal = np.empty(betaTab.size)
immunisesFinal = np.empty(betaTab.size)
epidemie_endigue_list = np.empty(betaTab.size)
epidemie_endigue=False

for k, beta in enumerate(betaTab):

    R0 = beta / gamma * S_init
    R0List[k] = R0
    S = np.empty(n)
    I = np.empty(n)
    R = np.empty(n)
    y = np.empty(3)
    y[0] = S_init
    y[1] = I_init
    y[2] = R_init

    params = {"beta": beta, "gamma": gamma}
    model = mathUtils.SIR(params)
    deriv = model.deriv_SIR
    euler = model.euler

    for i in range(n):
        S[i] = y[0]
        I[i] = y[1]
```

```
        R[i] = y[2]
        Rt= S[i]*R0
        if not epidemie_endigue and Rt<=1  :
            epidemie_endigue = True
            epidemie_endigue_list[k]=R[i]

        t = temps[i]
        y = euler(t, dt, y, deriv)

    immunisesFinal[k] = 100 * R[-1]


plt.figure()
plt.title("% de la popatOn immunisée à la fin de l'épidemie en fctO de R0")
plt.plot(R0List, immunisesFinal)
plt.xlabel("R0")
plt.ylabel("Rt final")
plt.savefig("sir_Rtfinal_vs_R0.pdf", bbox_inches='tight')
plt.show()


# par virus

virus = [
    "Ebola",
    "Grippe saisonniere",
    "COVID initial",
    "COVID Delta",
    "COVID Omicron",
    "Variole",
    "Rougeole"
]

R0 = [
    1.95,   # Ebola
    1.27,   # grippe saisonniere
    2.79,   # COVID souche initiale
    5.08,   # Delta
    9.5,    # Omicron
    4.75,   # variole
    15.0    # rougeole
]

# DataFrame
df = pd.DataFrame({
    "Virus": virus,
    "R0": R0
})

display(df)

x = np.arange(len(virus))
plt.figure(figsize=(10, 5))
plt.bar(x, R0)
plt.xticks(x, virus, rotation=30)
plt.xlabel("Virus")
plt.ylabel("R0")
plt.title("Nombre de reproduction de base R0 selon le virus")

for i, v in enumerate(R0):
```

```
    plt.text(i, v + 0.2, str(v), ha="center")

plt.savefig("histogramme_R0_virus.pdf", bbox_inches='tight')
plt.show()


gamma = 0.2
dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

virus = [
    "MERS",
    "Grippe saisonniere",
    "Grippe H1N1 2009",
    "Ebola",
    "COVID initial",
    "Variole",
    "COVID Delta",
    "COVID Omicron",
    "Rougeole"
]

R0list = np.array([
    0.69,
    1.27,
    1.46,
    1.95,
    2.79,
    4.75,
    5.08,
    9.5,
    15.0
])

# beta = gamma * R0  (si S_init = 1)
betaTab = gamma * R0list

R0Final = np.empty(betaTab.size)
RFinal = np.empty(betaTab.size)
immunisesFinal = np.empty(betaTab.size)
epidemie_endigue_list = np.empty(betaTab.size)


for k, beta in enumerate(betaTab):
    epidemie_endigue=False
    R0 = beta / gamma * S_init
    R0Final[k] = R0

    S = np.empty(n)
    I = np.empty(n)
    R = np.empty(n)

    y = np.empty(3)
    y[0] = S_init
    y[1] = I_init
    y[2] = R_init

    params = {"beta": beta, "gamma": gamma}
```

```
    model = mathUtils.SIR(params)
    deriv = model.deriv_SIR
    euler = model.euler

    for i in range(n):
        S[i] = y[0]
        I[i] = y[1]
        R[i] = y[2]
        t = temps[i]
        y = euler(t, dt, y, deriv)
        Rt= S[i]*R0
        if not epidemie_endigue and Rt<=1  :
            epidemie_endigue = True
            epidemie_endigue_list[k]=(1-S[i])*100

    RFinal[k] = R[-1]
    immunisesFinal[k] = 100 * R[-1]

x = np.arange(len(virus))

labels = [
    f"{virus[i]}\n(R0={R0list[i]:.2f})"
    for i in range(len(virus))
]

plt.figure(figsize=(10, 5))
plt.bar(x, epidemie_endigue_list)

plt.xticks(x, labels, rotation=30)
plt.xlabel("Maladie (R0)")
plt.ylabel("Population immunisée")
plt.title("Population immunisée necessaire pour endiguer l'epidemie")

for i, v in enumerate(epidemie_endigue_list):
    plt.text(i, v + 1, f"{v:.1f}%", ha="center")

plt.savefig("histogramme_immunités.pdf", bbox_inches='tight')
plt.show()



plt.figure(figsize=(10, 5))
plt.bar(x, immunisesFinal)

plt.xticks(x, labels, rotation=30)
plt.xlabel("Maladie (R0)")
plt.ylabel("Population immunisée")
plt.title("Population immunisée à la fin de la maladie (sans confinement etc
    )")

for i, v in enumerate(immunisesFinal):
    plt.text(i, v + 1, f"{v:.1f}%", ha="center")

plt.savefig("histogramme_immunités fin de maladie", bbox_inches='tight')
plt.show()
```

Listing 2 – Code du modèle SIRDV couple

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```python
import nosPackages.mathUtils as mathUtils


# population initial classique
S_init=0.699 # personnes suscétibles (départ) en fraction -> 1=100%
I_init=0.001 # personnes infectés
R_init=0 # personnes guérit
Z_init=0 # décès

# population initial vulnerables
Sv_init=0.299 # personnes suscétibles (départ) en fraction -> 1=100%
Iv_init=0.001 # personnes infectés
Rv_init=0 # personnes guérit
Zv_init=0 # décès

# paramètre de contagion
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6 personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
R0=beta/gamma*S_init # nombre de personnes contaminés par un malade = 3 pour le covid

# paramètre de contagion vulnerable
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6 personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
betaV=0.558-0.1 # taux contagion pour vulnerable
gammaV=gamma/2 # 1/nb jour contagion vulnerable
crossbeta=0.3 # taux de contagion intergroupe
mu=0.002 # 1/ taux de decès
muV=mu*10 # taux de deces vulnerable
R0=beta/(gamma+mu)*S_init + crossbeta/(gamma+mu)*Sv_init # nombre de personnes contaminés par un malade = 3 pour le covid
R0v=betaV/(gammaV+muV)*Sv_init + crossbeta/(gammaV+muV)*S_init # nombre de personnes contaminés par un malade = 3 pour le covid

params = {
    "beta": [[beta, crossbeta],
             [crossbeta, betaV]],
    "gamma": gamma,
    "gamma_v": gammaV,
    "mu":mu,
    "mu_v": muV
}



dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

S = np.empty(n)
I = np.empty(n)
R = np.empty(n)
Z = np.empty(n)
Rt = np.empty(n)
Sv = np.empty(n)
Iv = np.empty(n)
Rv = np.empty(n)
```

```
Zv = np.empty(n)
Rtv = np.empty(n)

y = np.empty(8)
y[0] = S_init
y[1] = I_init
y[2] = R_init
y[3] = Z_init
y[4] = Sv_init
y[5] = Iv_init
y[6] = Rv_init
y[7] = Zv_init

model=mathUtils.SIR(params)
deriv=model.deriv_SIRD_2groupes_beta_matrix
euler=model.euler

for i in range(n):
    S[i] = y[0]
    I[i] = y[1]
    R[i] = y[2]
    Z[i] = y[3]
    Sv[i] = y[4]
    Iv[i] = y[5]
    Rv[i] = y[6]
    Zv[i] = y[7]
    t = temps[i]
    y = euler(t, dt, y, deriv)

Rt=beta/(gamma+mu)*S + crossbeta/(gamma+mu)*Sv # nombre de personnes contaminés par un malade = 3 pour le covid
Rtv=betaV/(gammaV+muV)*Sv + crossbeta/(gammaV+muV)*S # nombre de personnes contaminés par un malade vulnerable = 3 pour le covid

idx_fin = np.where(Rt < 1)[0]
jour_fin = temps[idx_fin[0]]
population_pour_immunite=S_init-S[idx_fin[0]]

idx_fin_v = np.where(Rtv < 1)[0]
jour_finV = temps[idx_fin_v[0]]
population_pour_immunite_v=Sv_init-Sv[idx_fin_v[0]]


fig, (ax1, ax3) = plt.subplots(1, 2, figsize=(14, 5))

# --- Graphe groupe standard ---
ax1.set_title("Groupe standard")
total=S_init+I_init
ax1.plot(temps, S/total*100, label="S(t)")
ax1.plot(temps, I/total*100, label="I(t)")
ax1.plot(temps, R/total*100, label="R(t)")
ax1.plot(temps, Z/total*100, label="Z(t)")
ax1.set_xlabel("temps")
ax1.set_ylabel("S(t), I(t), R(t), Z(t)")
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(temps, Rt, "--", label="Rt")
ax2.axhline(1, linestyle=":", label="Rt = 1")
ax2.set_ylabel("Rt")
```

```
ax2.legend(loc="center right")


# --- Graphe groupe vulnérable ---
ax3.set_title("Groupe vulnérable")
total_v=Sv_init+Iv_init
ax3.plot(temps, Sv/total_v*100, label="Sv(t)")
ax3.plot(temps, Iv/total_v*100, label="Iv(t)")
ax3.plot(temps, Rv/total_v*100, label="Rv(t)")
ax3.plot(temps, Zv/total_v*100, label="Zv(t)")
ax3.set_xlabel("temps")
ax3.set_ylabel("Sv(t), Iv(t), Rv(t), Zv(t)")
ax3.legend(loc="upper right")

ax4 = ax3.twinx()
ax4.plot(temps, Rtv, "--", label="Rtv")
ax4.axhline(1, linestyle=":", label="Rtv = 1")
ax4.set_ylabel("Rtv")
ax4.legend(loc="center right")


plt.suptitle("Evolution SIRD et nombre de reproduction effectif")
plt.tight_layout()
plt.savefig("sir_SIRD_Rt_evolution_2groupes.pdf", bbox_inches="tight")
plt.show()



import pandas as pd
import numpy as np
import matplotlib.pyplot as plt


# Tailles initiales des 2 groupes
pop_init = S_init + I_init
pop_init_v = Sv_init + Iv_init
pop_init_tot = pop_init + pop_init_v

# Statistiques finales par groupe
# % de chaque groupe dans la population totale
pct_pop_init = 100 * pop_init / pop_init_tot
pct_pop_init_v = 100 * pop_init_v / pop_init_tot

# % décédés dans chaque groupe à la fin
pct_deces = 100 * Z[-1] / pop_init
pct_deces_v = 100 * Zv[-1] / pop_init_v

# % sains (susceptibles restants) à la fin
pct_sain_fin = 100 * S[-1] / pop_init
pct_sain_fin_v = 100 * Sv[-1] / pop_init_v

# % immunisés (guéris) à la fin
pct_immun_fin = 100 * R[-1] / pop_init
pct_immun_fin_v = 100 * Rv[-1] / pop_init_v

# % immunisés au moment où Rt passe sous 1
pct_immun_seuil = 100 * population_pour_immunite / pop_init
pct_immun_seuil_v = 100 * population_pour_immunite_v / pop_init_v

# ========================
```

```python
# DataFrame global
# ========================
df_resume = pd.DataFrame({
    "Population": ["Standard", "Vulnérable"],
    "Pop_init (%)": [pct_pop_init, pct_pop_init_v],
    "Immunisés": [pct_immun_fin, pct_immun_fin_v],
    "Sains": [pct_sain_fin, pct_sain_fin_v],
    "Deces": [pct_deces, pct_deces_v],
    "tx_pour_immunité": [pct_immun_seuil, pct_immun_seuil_v],
    "Jour Rt<1": [jour_fin, jour_finV]
})

# Optionnel : arrondi pour affichage
df_resume = df_resume.round({
    "Pop_init (%)": 2,
    "Immunisés": 2,
    "Sains": 2,
    "Deces": 2,
    "tx_pour_immunité": 2,
    "Jour Rt<1": 2
})

display(df_resume)


# Colonnes à tracer en pourcentage
cols_pct = [
    "Pop_init (%)",
    "Immunisés",
    "Sains",
    "Deces",
    "tx_pour_immunité"
]

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))

# -------------------------
# Population standard
# -------------------------
val_std = df_resume.loc[df_resume["Population"] == "Standard", cols_pct].iloc[0]

ax1.bar(cols_pct, val_std)
ax1.set_title(
    f"Population standard\nJour Rt<1 = {df_resume.loc[df_resume['Population']=='Standard', 'Jour Rt<1'].iloc[0]:.2f}"
)
ax1.set_ylabel("Pourcentage (%)")
ax1.tick_params(axis="x", rotation=45)

for i, v in enumerate(val_std):
    ax1.text(i, v, f"{v:.2f}%", ha="center", va="bottom")

# -------------------------
# Population vulnérable
# -------------------------
val_v = df_resume.loc[df_resume["Population"] == "Vulnérable", cols_pct].iloc[0]

ax2.bar(cols_pct, val_v)
```

```
ax2.set_title(
    f"Population vulnérable\nJour Rt<1 = {df_resume.loc[df_resume['Population']=='Vulnérable', 'Jour Rt<1'].iloc[0]:.2f}"
)
ax2.set_ylabel("Pourcentage (%)")
ax2.tick_params(axis="x", rotation=45)

for i, v in enumerate(val_v):
    ax2.text(i, v, f"{v:.2f}%", ha="center", va="bottom")

plt.suptitle("Résumé statistique des deux populations")
plt.tight_layout()
plt.show()


import numpy as np
import matplotlib.pyplot as plt

# Statistiques en pourcentage à comparer
stats_pct = ["Pop_init (%)", "Immunisés", "Sains", "Deces", "tx_pour_immunité"]

# Valeurs pour chaque population
vals_std = df_resume.loc[df_resume["Population"] == "Standard", stats_pct].iloc[0].values
vals_vul = df_resume.loc[df_resume["Population"] == "Vulnérable", stats_pct].iloc[0].values

# Jours de passage sous 1
jour_std = df_resume.loc[df_resume["Population"] == "Standard", "Jour Rt<1"].iloc[0]
jour_vul = df_resume.loc[df_resume["Population"] == "Vulnérable", "Jour Rt<1"].iloc[0]

x = np.arange(len(stats_pct))
width = 0.35

fig, ax1 = plt.subplots(figsize=(12, 6))

# Barres groupées
bar1 = ax1.bar(x - width/2, vals_std, width, label="Standard")
bar2 = ax1.bar(x + width/2, vals_vul, width, label="Vulnérable")

ax1.set_title("Comparaison des statistiques : standard vs vulnérable")
ax1.set_ylabel("Pourcentage (%)")
ax1.set_xticks(x)
ax1.set_xticklabels(stats_pct, rotation=30, ha="right")
ax1.legend(loc="upper left")

# Valeurs au-dessus des barres
for b in bar1:
    h = b.get_height()
    ax1.text(b.get_x() + b.get_width()/2, h, f"{h:.2f}%", ha="center", va="bottom")

for b in bar2:
    h = b.get_height()
    ax1.text(b.get_x() + b.get_width()/2, h, f"{h:.2f}%", ha="center", va="bottom")

```

```
# Axe secondaire pour le jour Rt<1
ax2 = ax1.twinx()
ax2.set_ylabel("Jour Rt < 1")

# On place les deux points "jour" à droite du graphe
x_jour = len(stats_pct)
ax2.scatter(x_jour - width/2, jour_std, marker="o", s=80, label="Jour Rt<1
    Standard")
ax2.scatter(x_jour + width/2, jour_vul, marker="s", s=80, label="Jour Rt<1
    Vulnérable")

ax2.text(x_jour - width/2, jour_std, f"{jour_std:.2f}", ha="center", va="
    bottom")
ax2.text(x_jour + width/2, jour_vul, f"{jour_vul:.2f}", ha="center", va="
    bottom")

# Ajouter le libellé de cette dernière catégorie
xticks = list(x) + [x_jour]
xlabels = stats_pct + ["Jour Rt<1"]
ax1.set_xticks(xticks)
ax1.set_xticklabels(xlabels, rotation=30, ha="right")

# Fusion des légendes
h1, l1 = ax1.get_legend_handles_labels()
h2, l2 = ax2.get_legend_handles_labels()
ax1.legend(h1 + h2, l1 + l2, loc="upper center")

plt.tight_layout()
plt.savefig("sir_SIRD_Rt_stats_finales_2groupes.pdf", bbox_inches="tight")
plt.show()
```

Listing 3 – Code du modèle SIRDV

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nosPackages.mathUtils as mathUtils

# population initial
S_init=0.999 # personnes suscétibles (départ) en fraction -> 1=100%
I_init=0.001 # personnes infectés
R_init=0 # personnes guérissent
D_init=0 # personnes qui meurent

# paramètre de contagion
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6
    personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
mu= 0.01 # Coefficient de décès (valeur non réelle)
R0=beta/gamma*S_init # nombre de personnes contaminés par un malade = 3 pour
     le covid

dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

S = np.empty(n)
I = np.empty(n)
R = np.empty(n)
D = np.empty(n)
```

```
Rt = np.empty(n)

y = np.empty(4)
y[0] = S_init
y[1] = I_init
y[2] = R_init
y[3] = D_init

params={"beta":beta,"gamma":gamma, "mu":mu}
model=mathUtils.SIR(params)
deriv=model.deriv_SIRD
euler=model.euler

for i in range(n):
    S[i] = y[0]
    I[i] = y[1]
    R[i] = y[2]
    D[i] = y[3]
    t = temps[i]
    y = euler(t, dt, y, deriv)
Rt = R0*S # evolution du R0 (contagion pour une personne)


plt.figure()
plt.title("Evolution de S(t), I(t), R(t), D(t) et Rt")

ax1 = plt.gca()
ax1.plot(temps, S, label="S(t)")
ax1.plot(temps, I, label="I(t)")
ax1.plot(temps, R, label="R(t)")
ax1.plot(temps, D, label="D(t)")
ax1.set_xlabel("temps")
ax1.set_ylabel("S(t), I(t), R(t), D(t)")
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(temps, Rt, "--", label="Rt")
ax2.axhline(1, linestyle=":", label="Rt = 1")
ax2.set_ylabel("Rt")
ax2.legend(loc="center right")

plt.savefig("sir_SIRD_Rt.pdf", bbox_inches='tight')
plt.show()

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import nosPackages.mathUtils as mathUtils

# population initial
S_init=0.999 # personnes suscétibles (départ) en fraction -> 1=100%
I_init=0.001 # personnes infectés
R_init=0 # personnes guérissent
D_init=0 # personnes qui meurent
V_init=0 # personnes vaccinées

# paramètre de contagion
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6
    personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
```

```
mu= 0.002 # Coefficient de décès (valeur non réelle)
omega= 0.01 # Taux de personnes vaccinées
R0=beta/(gamma)*S_init # nombre de personnes contaminés par un malade = 3
    pour le covid
dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

S = np.empty(n)
I = np.empty(n)
R = np.empty(n)
D = np.empty(n)
V = np.empty(n)
Rt = np.empty(n)

y = np.empty(5)
y[0] = S_init
y[1] = I_init
y[2] = R_init
y[3] = D_init
y[4] = V_init

params={"beta":beta, "gamma":gamma, "mu":mu, "omega":omega}
model=mathUtils.SIR(params)
deriv=model.deriv_SIRDV
euler=model.euler

for i in range(n):
    S[i] = y[0]
    I[i] = y[1]
    R[i] = y[2]
    D[i] = y[3]
    V[i] = y[4]
    t = temps[i]
    y = euler(t, dt, y, deriv)
Rt = R0*S # evolution du R0 (contagion pour une personne)
plt.figure()
plt.title("Evolution de S(t), I(t), R(t), D(t), V(t) et Rt")

ax1 = plt.gca()
ax1.plot(temps, S, label="S(t)")
ax1.plot(temps, I, label="I(t)")
ax1.plot(temps, R, label="R(t)")
ax1.plot(temps, D, label="D(t)")
ax1.plot(temps, V, label="V(t)")
ax1.set_xlabel("temps")
ax1.set_ylabel("S(t), I(t), R(t), D(t), V(t)")
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(temps, Rt, "--", label="Rt")
ax2.axhline(1, linestyle=":", label="Rt = 1")
ax2.set_ylabel("Rt")
ax2.legend(loc="center right")

plt.savefig("sir_SIRDV_Rt.pdf", bbox_inches='tight')
plt.show()
```

```python


# population initial
S_init=0.999 # personnes suscétibles (départ) en fraction -> 1=100%
I_init=0.001 # personnes infectés
R_init=0 # personnes guérissent
D_init=0 # personnes qui meurent
V_init=0 # personnes vaccinées

# paramètre de contagion
beta=0.558 # taux de contagion journalier de base pour le covid = 0.6 personnes/jour
gamma=0.2 # 1/gamma = nb de jour contagieux = 5 jour pour le covid
mu= 0.002 # Coefficient de décès (valeur non réelle)
omega= 0.01 # Taux de personnes vaccinées
R0=beta/(gamma)*S_init # nombre de personnes contaminés par un malade = 3 pour le covid




dt = 0.1
tmax = 100.0
temps = np.arange(0, tmax + dt, dt)
n = temps.size

S = np.empty(n)
I = np.empty(n)
R = np.empty(n)
D = np.empty(n)
V = np.empty(n)
Rt = np.empty(n)

y = np.empty(5)
y[0] = S_init
y[1] = I_init
y[2] = R_init
y[3] = D_init
y[4] = V_init

params={"beta":beta, "gamma":gamma, "mu":mu, "omega":omega}
model=mathUtils.SIR(params)
deriv=model.deriv_SIRDV
euler=model.euler

for i in range(n):
    S[i] = y[0]
    I[i] = y[1]
    R[i] = y[2]
    D[i] = y[3]
    V[i] = y[4]
    t = temps[i]
    y = euler(t, dt, y, deriv)
Rt = R0*S # evolution du R0 (contagion pour une personne)




plt.figure()
```

```
plt.title("Evolution de S(t), I(t), R(t), D(t) et Rt")

ax1 = plt.gca()
ax1.plot(temps, S, label="S(t)")
ax1.plot(temps, I, label="I(t)")
ax1.plot(temps, R, label="R(t)")
ax1.plot(temps, D, label="D(t)")
ax1.plot(temps, V, label="V(t)")
ax1.set_xlabel("temps")
ax1.set_ylabel("S(t), I(t), R(t), D(t), V(t)")
ax1.legend(loc="upper right")

ax2 = ax1.twinx()
ax2.plot(temps, Rt, "--", label="Rt")
ax2.axhline(1, linestyle=":", label="Rt = 1")
ax2.set_ylabel("Rt")
ax2.legend(loc="center right")

plt.savefig("sir_SIRDV_Rt.pdf", bbox_inches='tight')
plt.show()
```

Listing 4 – code Methodes d'euler et dérives

```
import numpy as np

class SIR:

    def __init__(self,params:dict[str,float]):
        self.params=params

    def rk4(x, dx, y, deriv,params):
        """
        /*----------------------------------------
        sous programme de resolution d'equations
        differentielles du premier ordre par
        la methode de Runge-Kutta d'ordre 4
        x = abscisse, une valeur scalaire, par exemple le temps
        dx = pas, par exemple le pas de temps
        y = valeurs des fonctions au temps t(i), c'est un tableau numpy de
    taille n
        avec n le nombre d'équations différentielles du 1er ordre

        rk4 renvoie les nouvelles valeurs de y pour t(i+1)

        deriv = variable contenant le nom du
        sous-programme qui calcule les derivees
        deriv doit avoir trois arguments: deriv(x,y,params) et renvoyer
        un tableau numpy dy de taille n
        ----------------------------------------*/
        """
        #  /* d1, d2, d3, d4 = estimations des derivees
        #    yp = estimations intermediaires des fonctions */
        ddx = dx/2.       #         /* demi-pas */
        d1 = deriv(x,y,params)   #       /* 1ere estimation */
        yp = y + d1*ddx
        #    for  i in range(n):
        #        yp[i] = y[i] + d1[i]*ddx
        d2 = deriv(x+ddx,yp,params)     #/* 2eme estimat. (1/2 pas) */
        yp = y + d2*ddx
        d3 = deriv(x+ddx,yp,params)  #/* 3eme estimat. (1/2 pas) */
        yp = y + d3*dx
```

```
        d4 = deriv(x+dx,yp,params)     #  /* 4eme estimat. (1 pas) */
        #/* estimation de y pour le pas suivant en utilisant
        #  une moyenne ponderee des derivees en remarquant
        #  que : 1/6 + 1/3 + 1/3 + 1/6 = 1 */
        return y + dx*( d1 + 2*d2 + 2*d3 + d4 )/6

    def deriv_SIR(self,t,y,params):
        """
        Derivees pour SIR
        """
        beta = params["beta"]
        gamma= params["gamma"]
        dy = np.zeros(3)
        dy[0] = -beta*y[0]*y[1] # dérivée de S
        dy[1] = beta*y[0]*y[1]-gamma*y[1] # dérivée de I
        dy[2] = gamma*y[1] # dérivée de R

        return dy

    def deriv_SIRD_2groupes_beta_matrix(self, t, y, params):
        """
            Derivees pour SIRD
        """
        s, i, r, d, s_v, i_v, r_v, d_v = y

        beta = np.array(params["beta"])
        # beta = [[beta_JJ, beta_JV],
        #         [beta_VJ, beta_VV]]

        gamma = params["gamma"]
        gamma_V = params["gamma_v"]
        mu = params["mu"]
        mu_V = params["mu_v"]

        dy = np.zeros(8)

        # Forces d'infection
        lambda_S = beta[0, 0] * i + beta[0, 1] * i_v
        lambda_V = beta[1, 0] * i + beta[1, 1] * i_v

        # Groupe des non vulnérables
        dy[0] = -s * lambda_S
        dy[1] = s * lambda_S - (gamma + mu) * i
        dy[2] = gamma * i
        dy[3] = mu * i

        # Groupe vulnérables
        dy[4] = -s_v * lambda_V
        dy[5] = s_v * lambda_V - (gamma_V + mu_V) * i_v
        dy[6] = gamma_V * i_v
        dy[7] = mu_V * i_v

        return dy

    def deriv_SIRD(self, t, y, params):
        """
        Derivees pour SIRD
        """
        beta = params["beta"]
        gamma= params["gamma"]
```

```
        mu= params["mu"]
        dy = np.zeros(4)
        dy[0] = -beta*y[0]*y[1] # dérivée de S
        dy[1] = beta*y[0]*y[1]-gamma*y[1] - mu*y[1] # dérivée de I
        dy[2] = gamma*y[1] # dérivée de R
        dy[3] = mu*y[1] # Dérivée de D

        return dy

    def deriv_SIRD_dev(self, t, y, params):
        """
        Derivees pour SIR
        """
        beta = params["beta"]
        gamma= params["gamma"]
        mu= params["mu"]
        rho= params["rho"]
        dy = np.zeros(4)
        dy[0] = -beta*y[0]*y[1] + rho*y[2] # dérivée de S
        dy[1] = beta*y[0]*y[1]-gamma*y[1] - mu*y[1] # dérivée de I
        dy[2] = gamma*y[1] - rho*y[2] # dérivée de R
        dy[3] = mu*y[1] # Dérivée de D

        return dy

    def deriv_SIRDV(self, t, y, params):
        """
        Derivees pour SIR
        """
        beta = params["beta"]
        gamma= params["gamma"]
        mu = params["mu"]
        omega = params["omega"]
        dy = np.zeros(5)
        dy[0] = -beta*y[0]*y[1] - omega*y[0] # dérivée de S
        dy[1] = beta*y[0]*y[1] -gamma*y[1] - mu*y[1] # dérivée de I
        dy[2] = gamma*y[1] # dérivée de R
        dy[3] = mu*y[1] # Dérivée de D
        dy[4] = omega*y[0] # Dérivée de V

        return dy

    def euler(self,t, dt, y, deriv):
        """
        Un pas de Euler : y(t+dt) = y(t) + dt*y'(t)
        """
        params=self.params
        dy = deriv(t, y, params)
        y[:] = y + dt*dy
        return y

    def laplacien_neumann(self, I, h):
        """
        Calcule le laplacien discret 2D de la matrice I
        avec condition au bord sans flux (Neumann)
        """

        Ipad = np.pad(I, ((1, 1), (1, 1)), mode="edge")

        L = (
```

```
            Ipad[2:, 1:-1] +
            Ipad[:-2, 1:-1] +
            Ipad[1:-1, 2:] +
            Ipad[1:-1, :-2] -
            4 * Ipad[1:-1, 1:-1]
        ) / h**2

        return L

    def deriv_SIRD_euler_explicite_spatiale(self, t, y, params):
        """
            Derivees pour SIRD
        """
        s, i, r, d = y

        beta = params["beta"]
        gamma = params["gamma"]
        mu = params["mu"]
        D = params["D"]
        h = params["h"]
        dy = np.zeros_like(y)

        # Groupe des non vulnérables
        l=self.laplacien_neumann(i,h)
        dy[0] = -s * beta
        dy[1] = s * beta - (gamma + mu) * i +D*l
        dy[2] = gamma * i
        dy[3] = mu * i

        return dy

    def euler(self,t, dt, y, deriv):
        """
        Un pas de Euler : y(t+dt) = y(t) + dt*y'(t)
        """
        params=self.params
        dy = deriv(t, y, params)
        y[:] = y + dt*dy
        return y

class COVID19:

    def __init__(self,params:dict[str,float]):
        self.params=params

    def rk4(x, dx, y, deriv,params):
        """
        /*----------------------------------------
        sous programme de resolution d'equations
        differentielles du premier ordre par
        la methode de Runge-Kutta d'ordre 4
        x = abscisse, une valeur scalaire, par exemple le temps
        dx = pas, par exemple le pas de temps
        y = valeurs des fonctions au temps t(i), c'est un tableau numpy de taille n
        avec n le nombre d'équations différentielles du 1er ordre

        rk4 renvoie les nouvelles valeurs de y pour t(i+1)

        deriv = variable contenant le nom du
```

```
        sous-programme qui calcule les derivees
        deriv doit avoir trois arguments: deriv(x,y,params) et renvoyer
        un tableau numpy dy de taille n
        ----------------------------------------*/
        """
        #  /* d1, d2, d3, d4 = estimations des derivees
        #    yp = estimations intermediaires des fonctions */
        ddx = dx/2.       #         /* demi-pas */
        d1 = deriv(x,y,params)   #       /* 1ere estimation */
        yp = y + d1*ddx
        #    for  i in range(n):
        #        yp[i] = y[i] + d1[i]*ddx
        d2 = deriv(x+ddx,yp,params)     #/* 2eme estimat. (1/2 pas) */
        yp = y + d2*ddx
        d3 = deriv(x+ddx,yp,params)  #/* 3eme estimat. (1/2 pas) */
        yp = y + d3*dx
        d4 = deriv(x+dx,yp,params)     #  /* 4eme estimat. (1 pas) */
        #/* estimation de y pour le pas suivant en utilisant
        #  une moyenne ponderee des derivees en remarquant
        #  que : 1/6 + 1/3 + 1/3 + 1/6 = 1 */
        return y + dx*( d1 + 2*d2 + 2*d3 + d4 )/6

    def model(y, t, beta, gamma, mu, nu_max, t_start):
        S, I, R, D, V = y
        N = S + I + R + D + V

        # Condition de déclenchement
        if t < t_start:
            nu = 0
        else:
            nu = nu_max

        dSdt = -beta * S * I / N - nu * S
        dIdt = beta * S * I / N - (gamma + mu) * I
        dRdt = gamma * I
        dDdt = mu * I
        dVdt = nu * S

        return [dSdt, dIdt, dRdt, dDdt, dVdt]

    def deriv(self, t, y, params):
        """
        Derivees pour SIR
        """
        beta = params["beta"]
        gamma = params["gamma"]
        mu = params["mu"]
        omega_max = params["omega"]
        rho = params["rho"]
        t_start = params["t_start"]

        # CONDITION DE DÉBUT DE VACCINATION
        if t < t_start :
            omega = 0
        else:
            omega = omega_max

        dy = np.zeros(5)
        dy[0] = -beta*y[0]*y[1] - omega*y[0] + rho*y[4] + rho*y[2] # dérivée
    de S
```

```
        dy[1] = beta*y[0]*y[1] -gamma*y[1] - mu*y[1] # dérivée de I
        dy[2] = gamma*y[1] - rho*y[2] # dérivée de R
        dy[3] = mu*y[1] # Dérivée de D
        dy[4] = omega*y[0] - rho*y[4] # Dérivée de V

        return dy

    def euler(self,t, dt, y, deriv):
        """
        Un pas de Euler : y(t+dt) = y(t) + dt*y'(t)
        """
        params=self.params
        dy = deriv(t, y, params)
        y[:] = y + dt*dy
        return y

class ZOMBIE:

    def __init__(self,params:dict[str,float]):
        self.params=params

    def rk4(x, dx, y, deriv,params):
        """
        /*----------------------------------------
        sous programme de resolution d'equations
        differentielles du premier ordre par
        la methode de Runge-Kutta d'ordre 4
        x = abscisse, une valeur scalaire, par exemple le temps
        dx = pas, par exemple le pas de temps
        y = valeurs des fonctions au temps t(i), c'est un tableau numpy de taille n
        avec n le nombre d'équations différentielles du 1er ordre

        rk4 renvoie les nouvelles valeurs de y pour t(i+1)

        deriv = variable contenant le nom du
        sous-programme qui calcule les derivees
        deriv doit avoir trois arguments: deriv(x,y,params) et renvoyer
        un tableau numpy dy de taille n
        ----------------------------------------*/
        """
        #  /* d1, d2, d3, d4 = estimations des derivees
        #    yp = estimations intermediaires des fonctions */
        ddx = dx/2.       #         /* demi-pas */
        d1 = deriv(x,y,params)   #       /* 1ere estimation */
        yp = y + d1*ddx
        #    for  i in range(n):
        #        yp[i] = y[i] + d1[i]*ddx
        d2 = deriv(x+ddx,yp,params)     #/* 2eme estimat. (1/2 pas) */
        yp = y + d2*ddx
        d3 = deriv(x+ddx,yp,params)  #/* 3eme estimat. (1/2 pas) */
        yp = y + d3*dx
        d4 = deriv(x+dx,yp,params)     #  /* 4eme estimat. (1 pas) */
        #/* estimation de y pour le pas suivant en utilisant
        #  une moyenne ponderee des derivees en remarquant
        #  que : 1/6 + 1/3 + 1/3 + 1/6 = 1 */
        return y + dx*( d1 + 2*d2 + 2*d3 + d4 )/6

    def deriv(self, t, y, params):
        """
```

```
        Derivees pour SIR
        """
        beta = params["beta"]
        gamma = params["gamma"]
        mu = params["mu"]
        rho = params["rho"]

        dy = np.zeros(5)
        dy[0] = -beta*y[0]*y[1] - omega*y[0] + rho*y[4] + rho*y[2] # dérivée de S
        dy[1] = beta*y[0]*y[1] -gamma*y[1] - mu*y[1] # dérivée de I
        dy[2] = gamma*y[1] - rho*y[2] # dérivée de R
        dy[3] = mu*y[1] # Dérivée de D
        dy[4] = omega*y[0] - rho*y[4] # Dérivée de V

        return dy

    def deriv_ZOMBIE(self, t, y, params):
        """
        Derivees pour SIRD
        """
        beta = params["beta"]
        gamma= params["gamma"]
        mu= params["mu"]
        alpha= params["alpha"]
        dy = np.zeros(4)
        dy[0] = - mu*y[0]*y[1] - beta*y[0]*y[1] # dérivée de S
        dy[1] = gamma*y[2] - alpha*y[0]*y[1] # dérivée de I
        dy[2] = -gamma*y[2] + beta*y[0]*y[1] # dérivée de R
        dy[3] = alpha*y[0]*y[1] + mu*y[0]*y[1] # Dérivée de D

        return dy

    def euler(self,t, dt, y, deriv):
        """
        Un pas de Euler : y(t+dt) = y(t) + dt*y'(t)
        """
        params=self.params
        dy = deriv(t, y, params)
        y[:] = y + dt*dy
        return y
```

Références

[1] Accromaths. Les modèles compartimentaux, 2023. Consulté le 12 avril 2026.

[2] Wikiland. Sir model, 2026. Consulté le 12 avril 2026.

[3] INRIA. Détails de la simulation épidémiologique, 2026. Consulté le 12 avril 2026.

[4] Philip Munz, Ioan Hudea, Joe Imad, and Robert J Smith. When zombies attack ! : mathematical modelling of an outbreak of zombie infection. *Infectious disease modelling research progress*, 4 :133–150, 2009.

[5] Evelyne Kolb Jean-Christophe Raut, Philippe Marcq. *Transport de matière*, chapter 9, pages 95–112. Université de la Sorbonne, Paris, France, 2025.